Exercise 9 - 12./13. 01. 2026

Learning Goals

We want to practice state management using simple Angular strategies like a singleton service. Using Signals we also get rid of complicated subscriptions. To further avoid complexity and inconsistent state, we will use computed() signals. We will also practice using effect() which is replacing subscriptions to handle side effects if a signal value changes.
Do not use RxJS for any of the exercises!

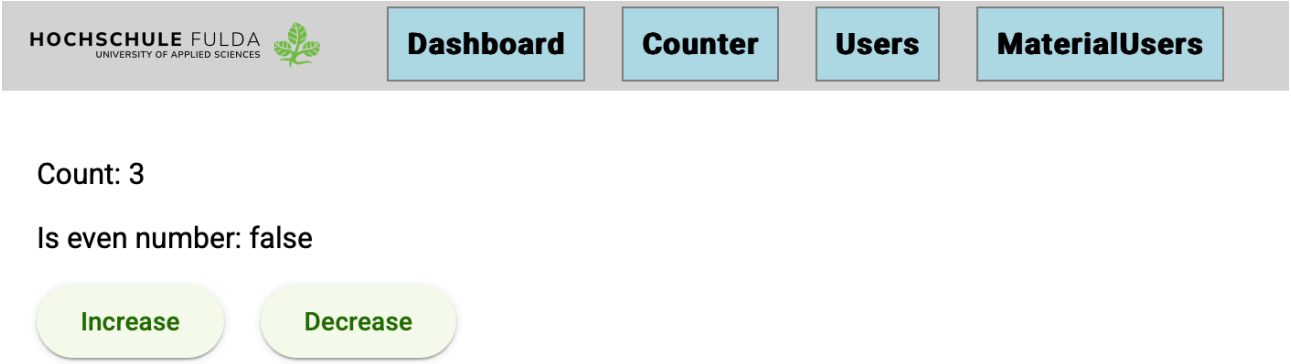
All about signals: https://angular.dev/guide/signals

(We will practice E2E tests (mentioned in exercise class 8) in exercise class 10).

Practical Exercises

1) Reuse your existing **Angular app** and create a CounterComponent which is linked in the header. The component has two buttons to trigger the increase or decrease of a numeric value by 1. Display the „count“ state value in the UI. Implement the counts as a WritableSignal<number>. Add an additional **computed** (=derived) Signal<boolean> „isEven“ that shows if the count is an even number. Use those in the template.
   From which Angular package do you need to import *{computed, Signal, signal, WritableSignal}* ?

   The UI result might look like this:

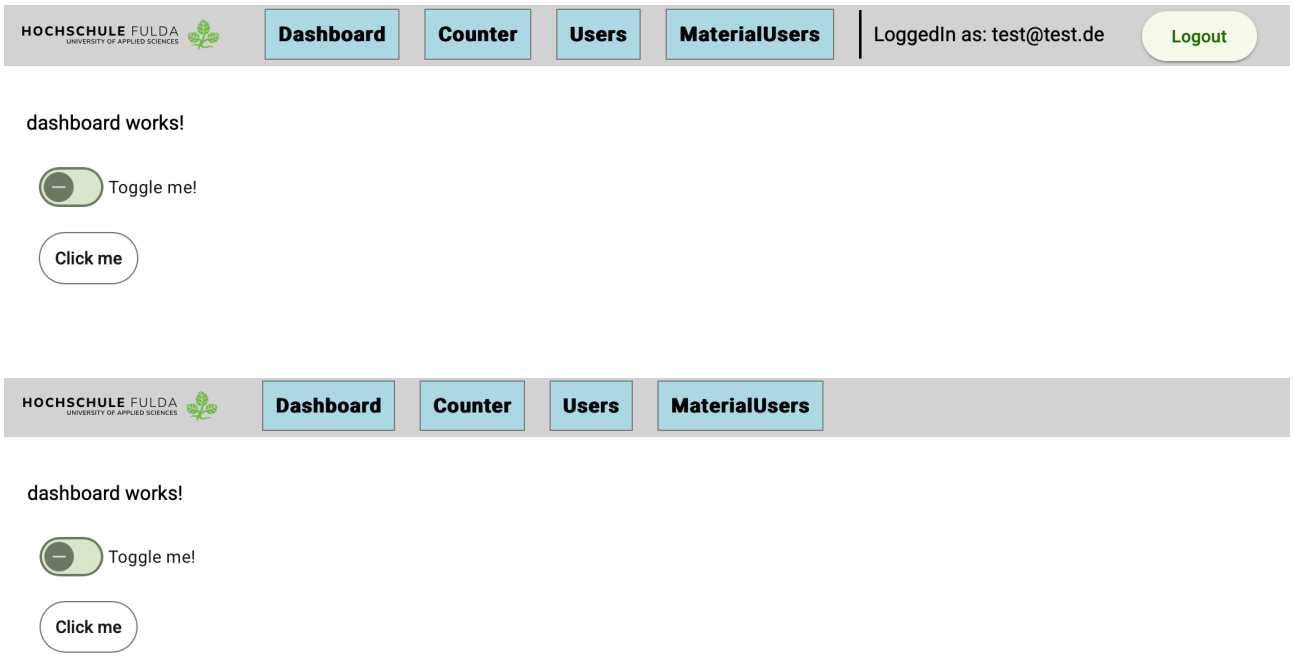


2) We implemented a login form in exercise class 8 within a ***Login(Form)Component*** which is reachable via /login. We want to create a singleton ***AppStateService*** (‚providedIn: root') which is shared among the ***Login(Form)Component*** and the ***HeaderComponent*** to be able to display whether we are logged in and who is the logged in user (if logged in).
Make sure neither LoginComponent nor HeaderComponent are standalone and create an ***AppModule*** (if you do not have one yet). Check the slides of lecture 5 how to refactor a standalone only app to one bootstrapping a module. In that module, you can provide a singleton service and declare components (like AppComponent, LoginComponent, HeaderComponent). It is possible to have a mix of standalone and non standalone components.

   The „onSubmit()" method called by the login form submission previously only set the rememberMe value at localStorage and redirected to the dashboard. Now we want to keep track of the ***currentUser*** and if the user ***isLoggedIn*** as global state values in form of Signals which are maintained by the AppStateService. Again, we want to avoid inconsistencies. So the ***isLoggedIn*** shall be a computed value based on the condition if the ***currentUser*** is set or not. Upon form submission, a login() method providing the value of ***currentUser*** as the eMail address is called. If no login happened yet, as default otherwise that value shall be an empty string. Connect the Login(Form)Component with the AppStateService to implement that logic. We will modify the HeaderComponent soon.

3) Now add the information about the ***currentUser*** to the HeaderComponent (if ***isLoggedIn*** is true) and read the value from the signal of the AppStateService. If ***isLoggedIn*** is true, also display a logout button in the header template. Using this approach with the two separate state values simplifies the logic needed in the HeaderComponent. Clicking on logout shall call a logout() method to update the signals at the global ***AppStateService***. After logout(), both user information and logout button shall disappear in the header.

The UI shall look similar to this:



4) The LoginForm contains a rememberMe value as a checkbox form field. We stored that value as a string in the ***localStorage*** but not in the ***AppStateService***. To avoid multiple deviating state informations, we want to keep track of this state value by adding a signal to the ***AppStateService***. The ***WritableSignal<boolean> rememberMe*** of the ***AppStateService*** shall be initialized using the value from the localStorage, if that is equal to ‚true'. Therefore the localStorage acts as persistence beyond the lifetime of the

Angular app, but we want to keep track of the value within Angular using signals. And we want to simplify state logic as much as possible in the Login(Form)Component.
Initialize the ***rememberMe*** value in the form of the Login(Form)Component with the initial ***rememberMe()*** signal value from the AppStateService.
Now refactor the onSubmit method of the Login(Form)Component to ***.set()*** a new value for the ***rememberMe*** signal owned by the AppStateService.
Instead of saving the value to the localStorage in the Login(Form)Component, we now want to hide that state synchronisation complexity in the AppStateService. Remove the code from the Login(Form)Component that interacts with localStorage.

Instead we want to use ***effect()*** to watch changes to the ***rememberMe*** signal in the ***AppStateService*** and keep the localStorage information in sync as a side effect. The code snippet will only be called upon updates to the rememberMe signal (or any additional signals referenced within the effect():

```
constructor() {
  effect(() => {
    console.log('Update - Remember me ' + this.rememberMe());
    localStorage.setItem('rememberMe', '' +
this.rememberMe());
  });
}
```

Visit the Login screen a couple of times and check the values in the Browser Debug Tools for localStorage to see that the new functionality works.